Refactor Prompt Generation into a Modular Creative Direction Engine

Objective

Redesign the image prompt system from a single string builder into a modular creative direction pipeline.

The goal is to make the application responsible for all creative decisions (fashion, styling, scene, lighting, photographer, etc.) while FLUX.2 Edit is responsible only for rendering those decisions and preserving the subject's identity.

Current Architecture

Current implementation:

```
Reference Image
        │
        ▼
Gemini Analysis
        │
        ▼
buildPrompt()
        │
        ▼
FLUX.2 Edit
```

The prompt currently consists of:

- Identity preservation
- Camera settings
- Genre string
- Mood

Problems:

- Identity instructions are repeated multiple times.
- Genre mappings are generic and cliché.
- No structured creative direction.
- Fashion is left to the model.
- Scene cannot be swapped independently.
- Difficult to extend or maintain.

New Architecture

Refactor to the following pipeline:

```
Reference Image
        │
        ▼
Gemini Analysis
        │
        ▼
Creative Direction Engine
        │
        ├── Identity Module
        ├── Artist Persona Module
        ├── Fashion Module
        ├── Scene Module
        ├── Pose Module
        ├── Photographer Module
        ├── Lighting Module
        ├── Rendering Quality Module
        │
        ▼
Prompt Composer
        │
        ▼
FLUX.2 Edit
```

Each module should be independent and reusable.

1. Standardize Gemini Output

Gemini should return structured JSON instead of descriptive text.

Example:

```
interface SubjectAnalysis {
  face: {
    shape: string;
    jaw: string;
    skinTone: string;
    undertone: "warm" | "cool" | "neutral";
```

```
    hair: string;
    facialHair?: string;
  };

  body: {
    build: string;
    shoulders: string;
    height: string;
  };

  vibe: {
    confidence: number;
    expression: string;
    perceivedAge: number;
  };
}
```

This becomes the input to every downstream module.

---

2. Identity Module

Keep identity instructions short.

Instead of repeating multiple "preserve identity" statements, use one concise section.

Example:

```
Keep the subject visually identical to the reference image.
Preserve facial identity, hairstyle, body proportions, age, skin tone, and
distinguishing features.
```

---

3. Replace Genre Prompts with Artist Archetypes

Genres should not directly control styling.

Instead create reusable creative archetypes.

Examples:

- Dark Luxury
- Luxury Rap

- Street Avant
- Minimal Pop
- Retro Soul
- Cyber Future
- Experimental Editorial

An archetype defines:

- overall mood
- fashion language
- visual tone
- atmosphere
- editorial style

Genres can map to one or more archetypes, but archetypes drive the visuals.

---

4. Fashion Rules Engine

The application should determine clothing based on Gemini analysis.

Examples:

```
Broad shoulders
→ cropped jackets

Athletic build
→ tailored trousers

Warm undertone
→ gold jewelry

Cool undertone
→ silver jewelry

Oval face
→ layered necklaces

Square face
→ open collars
```

The Fashion Module should output structured wardrobe pieces instead of prose.

Example:

```
Premium cropped leather jacket
Heavyweight cotton shirt
Relaxed tailored trousers
Chelsea boots
Layered gold jewelry
```

Do not rely on FLUX to invent fashion.

---

5. Scene Module (First-Class Feature)

Scenes must be completely independent of the artist, fashion, and persona.

A scene should be replaceable without changing anything else.

Example:

```
const scene = ScenePresets.WOODS;
```

or

```
const scene = ScenePresets.LUXURY_STUDIO;
```

or

```
const scene = ScenePresets.ROOFTOP;
```

Changing the scene should **only** change the environment.

It should **not** automatically change clothing.

Example:

Dark luxury artist

Fashion:

- black leather coat
- silver jewelry

Scene A:

- architectural recording studio

Scene B:

- misty evergreen forest

Scene C:

- rainy Tokyo street

Only the environment changes.

Scene Preset Structure

Represent scenes as structured data.

Example:

```
interface ScenePreset {
  name: string;

  location: string;

  atmosphere: string[];

  props: string[];

  lightingHints: string[];

  colorPalette: string[];

  cameraSuggestions: string[];
}
```

Example:

```
export const WOODS: ScenePreset = {
  name: "woods",

  location: "dense evergreen forest",

  atmosphere: [
```

```
    "morning mist",
    "cinematic fog",
    "quiet"
  ],

  props: [
    "ferns",
    "fallen logs",
    "moss"
  ],

  lightingHints: [
    "soft sun rays",
    "diffused daylight"
  ],

  colorPalette: [
    "deep green",
    "earth brown",
    "soft gold"
  ],

  cameraSuggestions: [
    "medium full body",
    "slightly low angle"
  ]
};
```

The Prompt Composer converts this into natural language.

---

6. Pose Module

Create reusable pose presets.

Examples:

- relaxed shoulders
- natural asymmetry
- direct eye contact
- effortless confidence
- editorial posture

This should not be embedded inside scene or photographer modules.

---

7. Photographer Module

Encapsulate photography decisions.

Responsible for:

- camera
- lens
- framing
- composition
- editorial quality

Example:

```
ARRI Alexa 65
85mm lens
Luxury magazine editorial
Natural skin texture
Filmic color science
```

---

8. Lighting Module

Independent lighting presets.

Examples:

Studio:

- large soft key
- subtle rim light

Outdoor:

- soft diffused sunlight
- volumetric rays

Night:

- practical lights
- neon reflections

Lighting should adapt to the selected scene but remain its own module.

---

9. Rendering Quality Module

Centralize rendering instructions.

Example:

```
Natural skin pores
Organic hair strands
Premium material textures
Realistic stitching
Fabric folds
Accurate jewelry reflections
Photorealistic
Filmic color grading
```

---

10. Prompt Composer

Replace string concatenation with a modular composer.

Example:

```
composer
  .add(identity)
  .add(persona)
  .add(fashion)
  .add(scene)
  .add(pose)
  .add(photographer)
  .add(lighting)
  .add(quality)
  .build();
```

Each module should generate its own text and remain independently testable.

Design Principles

1. Gemini analyzes the person.
2. The application decides the creative direction.
3. FLUX renders the final vision.

The application—not the model—should determine:

- wardrobe
- accessories
- scene
- lighting
- pose
- photographer style
- editorial direction

FLUX.2 Edit should primarily execute those instructions while preserving the subject's identity.

Long-Term Benefits

- Better image quality.
- More consistent outputs.
- Easier to add new scenes.
- Easier to add new artist archetypes.
- Fashion decisions become deterministic.
- Scene changes become one-line swaps.
- Prompt generation becomes modular and maintainable.
- Future features (album covers, music videos, press kits, tour posters) can reuse the same creative direction engine without rewriting prompt logic.